Lehrstuhl für Software und Systems Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

Agentic AI for Code Generation

Seminar Report

Student: Thua Duc Nguyen ✉ **and Supervisor: Derui Zhu** ✉

TUM School of Computation, Information and Technology, Technical University of Munich

✉ thuaduc.nguyen@tum.de
✉ derui.zhu@tum.de

June 30, 2026

Abstract — Large language models have moved from autocompleting single functions to autonomously resolving real software issues end to end, a shift popularly termed "vibe coding." This report surveys autonomous AI agents for code generation through a single unifying lens: a five-stage *vibe coding pipeline* (intent capture, planning, code generation, execution, and refinement). Using this framing we synthesise three active research fronts. First, we survey end-to-end software-engineering agents (SWE-agent, Agentless, AutoCodeRover, OpenHands, Devin) and the architectural patterns they share: the observe–think–act loop, agent–computer interfaces, code-as-action, and repository-scale context management, together with their evaluation on SWE-bench and its variants. Second, we examine planning and search strategies, from chain/tree/graph-of-thought reasoning to hierarchical localisation and sample-and-rank candidate selection. Third, we analyse execution feedback and iterative refinement, covering feedback signals, self-repair mechanisms, fault localisation, and convergence criteria. Across all three fronts we find recurring patterns: feedback-driven loops, execution-grounded selection, and a persistent context bottleneck, and we highlight open challenges in evaluation realism, cost, and trust. The pipeline framing exposes how planning and refinement, often studied in isolation, function as a coupled control layer over generation, but only when an *external* oracle closes the loop. Internal self-critique, we argue, is an open loop wearing a control loop's clothes; and intent capture, alone among the five stages, has no feedback mechanism at all.

1 Introduction

Motivation. Coding with a language model used to mean accepting an autocomplete suggestion. Increasingly it means describing a change in plain English and letting a system find the right files, edit them, run the tests, and iterate until they pass. The popular name for this style, "vibe coding" [20], captures both its appeal and the worry it raises: the developer states an intent and reviews an outcome, while the steps in between (reading the codebase, forming a hypothesis, running the debugger) are delegated to the model. The measured progress is real. In late 2023 the best system on SWE-bench, a benchmark of real GitHub issues, resolved under 2% of them [19]; two years later agents resolve most of the human-verified split. This report asks what produced that jump, and what it still does not deliver.

Background and Related Surveys. Existing surveys catalogue agents in software engineering [38] and LLM-based code-generation agents [10], organised largely by system or by capability. This report instead organises three active research fronts (end-to-end agents, planning and search, and execution-feedback refinement) around the pipeline stages they instantiate, which lets us compare systems that describe themselves in incompatible vocabularies.

Scope and Contribution. We cover agents whose output is a code change: repository-level issue resolution, automated program repair, and function-level synthesis, drawing on the 2023–2026 literature. Editor autocompletion, code search, and multi-agent role-play frameworks lie outside this scope. The report contributes, first, the vibe coding pipeline as a shared vocabulary for these systems (§2); second, a survey of the three fronts read against that pipeline (§3–§5); and third, a consolidation of what the fronts share and where they are jointly stuck (§6), drawing on 2025–2026 evidence about benchmark contamination, cost, and security that postdates most existing surveys.

2 The Vibe Coding Pipeline

2.1 What "Vibe Coding" Means Here

The term is Karpathy's, coined in February 2025 to describe a way of working in which one "fully give[s]

in to the vibes... and forget[s] that the code even exists" [20]. In that original sense vibe coding names a *human posture toward the output*: the developer states an intent, accepts the diff without reading it, and iterates on observed behaviour rather than on the source. It is a claim about what the human stops doing, not about how the machine is built.

We use the term in a deliberately broader, system-side sense. The *vibe coding pipeline* is the machinery that makes that posture available: the sequence of stages by which an agent turns an unread intent into a change it has itself validated. The generalisation is worth stating explicitly because it carries an obligation. If the developer no longer reads the diff, every guarantee the developer used to supply must be re-supplied by some stage of the pipeline. Which stages can actually supply such a guarantee, and which only appear to, is the question this report pursues. §6.4 and §6.5 return to Karpathy's narrow sense to argue that verification is precisely the guarantee the pipeline does *not* yet re-supply.

2.2 Five Stages

Papers on autonomous coding agents describe their systems in their own terms: phases, modules, roles, loops. Underneath, nearly all of them do the same five things. Naming those five stages gives us a frame for comparing systems that otherwise share no vocabulary. The pipeline is not a new architecture but a lens on existing ones.

Its stages echo the classical waterfall phases (requirements, design, implementation, testing), yet the resemblance is superficial. An agent traverses all five in seconds and repeats them many times per task, and the loop is closed by a test runner rather than by a review meeting. The closer analogue is the developer's inner loop of edit, run, and debug, with planning and code localisation folded into it.

The vibe coding pipeline (Figure 1) consists of five stages forming a loop: Intent Capture → Planning → Code Generation → Execution → Refinement. Execution feedback drives refinement, which may trigger re-planning or re-generation. Adjacent stages exchange structured artefacts: natural-language intent and repository context flow into plans and candidate patches; generated code is run against test oracles; failures and traces feed back into replanning or repair.

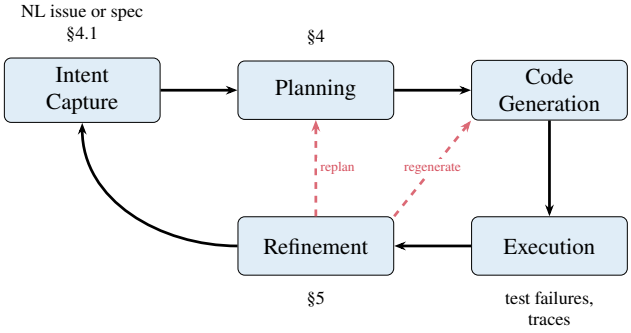

Figure 1 The vibe coding pipeline. Intent flows through planning, generation, and execution. Execution failures drive refinement, which either regenerates the patch or replans when the initial hypothesis is invalidated.

2.3 Mapping to Survey Topics

Each of the three main sections corresponds to one or more pipeline stages. §3 covers the full pipeline as an integrated system, including action interfaces for code generation (ACI, code-as-action, diff-based edits); §4 treats intent capture (§4.1) together with planning, candidate search, and the plan–generate interface, since in practice an agent operationalises intent and plans over it in the same breath; §5 covers execution environments, test-driven feedback, and the refinement loop (including fault-specific self-repair). Code generation is deliberately not given its own section: no surveyed system treats emitting tokens as a separable problem, and the design decisions that govern it (action space, edit format, candidate sampling) belong to the stages on either side of it. It is therefore distributed across §3 and §4. Cross-cutting concerns (evaluation, context management) are consolidated in the Discussion. Figure 2 provides a document-wide taxonomy: report sections mapped to pipeline stages, with representative systems and techniques as leaf exemplars.

3 End-to-End Autonomous Software Engineering Agents

Before examining individual pipeline stages, we survey systems that orchestrate the *entire* vibe coding pipeline, accepting a natural-language issue or specification and producing a tested code change with minimal human intervention. These end-to-end agents are the setting in which planning (§4) and feedback-driven refinement (§5) operate.

We begin by identifying the architectural patterns that underlie most current systems (§3.1). We then survey five representative agents that span the design

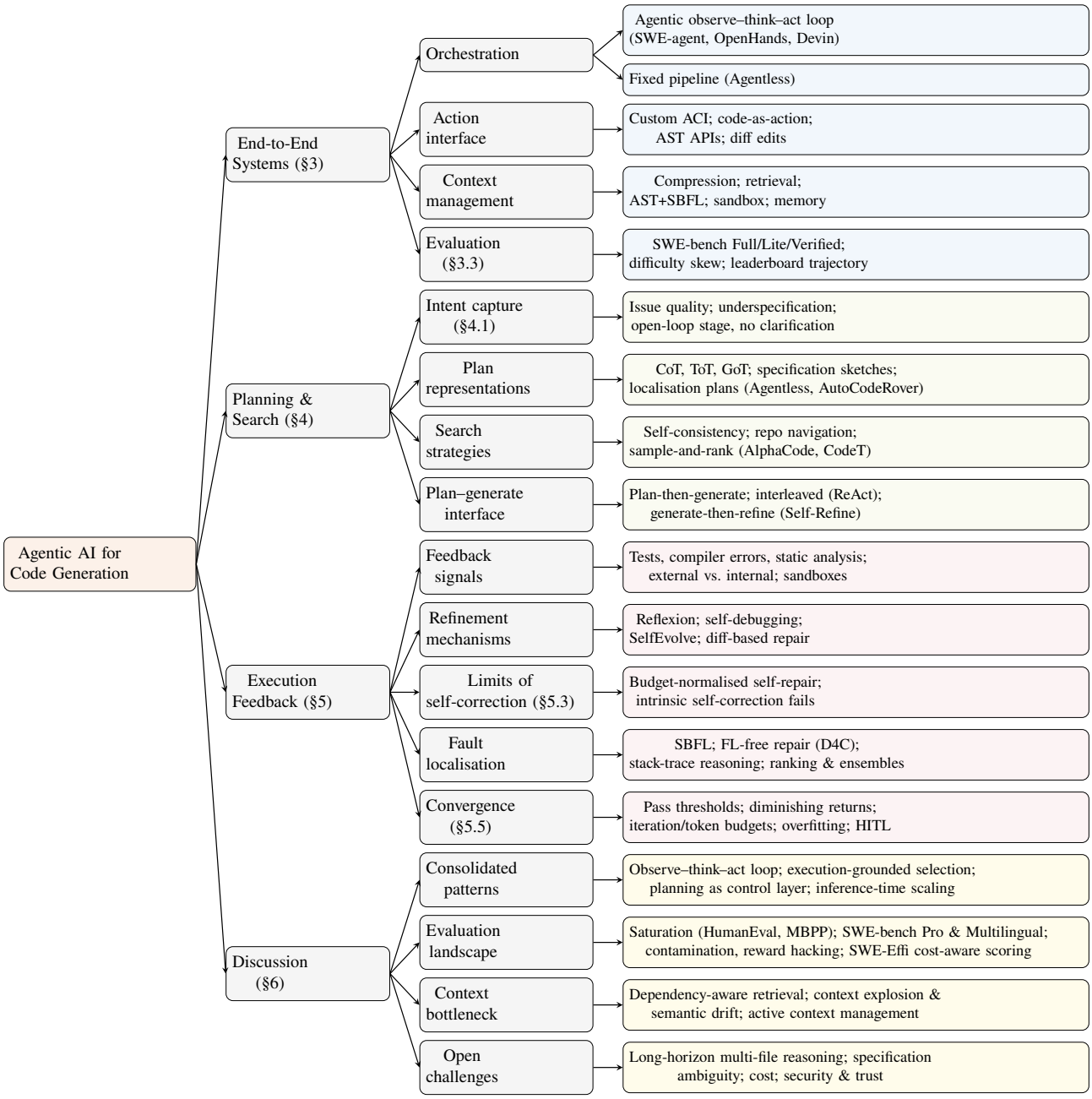

Figure 2 Survey taxonomy organised by report section and vibe coding pipeline stage. §3 surveys integrated end-to-end systems; §4 and §5 zoom into planning/search and execution-feedback mechanisms; §6 consolidates the recurring patterns, the evaluation landscape, the context bottleneck, and the open challenges. Leaf nodes name representative systems or techniques.

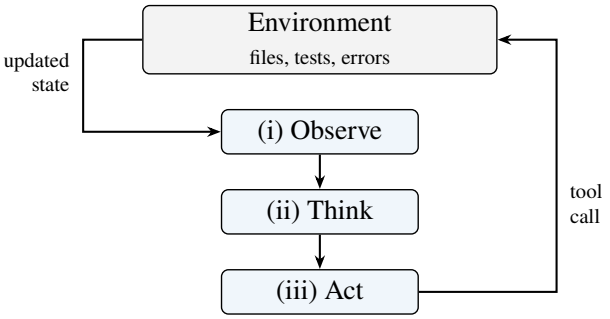

Figure 3 ReAct-style observe–think–act loop. The agent reads environment state, reasons about the next step, executes a tool call, and repeats.

space: from fully autonomous to deliberately non-agentic (§3.2). Finally, we examine how these systems are evaluated, focusing on SWE-bench and its variants (§3.3). This section instantiates the *End-to-End Systems* branch of Figure 2.

3.1 Architecture and Orchestration

Despite surface-level diversity, autonomous SE agents converge on a small number of architectural patterns. As illustrated in the *Orchestration*, *Action Interface*, and *Context Management* branches of Figure 2, these patterns determine how agents perceive their environment, what actions they can take, and how they manage repository-scale context.

The observe–think–act loop. The dominant paradigm is a ReAct-style [45] loop in which the agent alternates between (i) observing the environment (file contents, test output, error traces), (ii) reasoning about what to do next (chain-of-thought or structured plan), and (iii) executing an action via a tool call. SWE-agent [43] and OpenHands [36] both instantiate this loop, differing mainly in the *action space* exposed to the model (Figure 3).

Agent–Computer Interface (ACI). Yang et al. [43] introduce the ACI concept, arguing that LLMs are a "new category of end users" that need purpose-built interfaces rather than raw shell access. SWE-agent's ACI includes a windowed file viewer (100 lines), a linter-guarded `edit` command, summarised search results, and compressed observation history. Ablations show that each ACI component contributes measurably: removing the linter drops performance by 3 percentage points; switching to iterative search costs 6 pp; and showing full file content instead of a 100-line window costs 5.3 pp on SWE-bench Lite.

Code-as-action. OpenHands adopts the CodeAct paradigm [35], consolidating all agent actions into executable Python and bash code rather than a fixed tool API. Each turn may run arbitrary IPython cells, bash commands, or browser interactions inside a Docker-sandboxed environment. This design maximises flexibility at the cost of a larger action space that the model must navigate.

Context management. Repository-level tasks easily exceed LLM context windows. Agents employ several mitigation strategies: SWE-agent collapses older observations beyond the last 5 turns; AutoCodeRover [46] parses the codebase into an AST and exposes structure-aware search APIs (`search_class`, `search_method_in_class`) that return only signatures, not full definitions; Agentless [41] constructs a hierarchical repository skeleton and uses embedding-based retrieval to select candidate files before prompting the LLM.

3.2 Representative Systems

We survey five prominent systems that span the design space. Table 1 provides a comparative overview.

SWE-agent [43]. Developed at Princeton, SWE-agent wraps a GPT-4 Turbo backbone in a custom ACI with specialised file navigation, editing, and search commands. It achieved 12.47% on the full SWE-bench test set (2,294 tasks) and 18.00% on SWE-bench Lite when first published. The agent follows a typical trajectory: reproduce the issue, localise the faulty code via `find_file`/`search_dir`, then enter an edit–execute loop. Runs average $1.67 per instance on Lite, but the distribution is bimodal: successful trajectories finish in a median of 12 steps at $1.21, whereas failures burn a mean of 21 steps and $2.52 before giving up.

Agentless [41]. From UIUC, Agentless challenges the need for agentic autonomy. It replaces the interactive loop with a fixed three-phase pipeline: *localise* → *repair* → *validate*. Localisation uses a hierarchical narrowing strategy (files → classes/functions → edit locations) combining LLM prompting with embedding retrieval. Repair samples 42 candidate patches per bug in a search/replace diff format. Validation selects the best patch via regression testing, LLM-generated reproduction tests, and majority voting over AST-normalised

Table 1 Comparison of end-to-end autonomous SE agents. Architectural patterns: **OTA** = observe–think–act loop, **ACI** = custom Agent–Computer Interface, **CaA** = code-as-action, **Pipe** = fixed multi-phase pipeline (no agent loop). Splits are defined in Table 2; all scores are single-pass (pass@1) on each system's base configuration, so ablations and pass@k variants reported in the text are higher. Cost is the average USD per instance on Lite at pass@1. All figures are as-published by each system's own authors and reflect the backbone model of that publication, not current frontier models.

System	Architecture	Backbone	Context Strategy	Lite	Full	Cost
SWE-agent [43]	OTA + ACI	GPT-4 Turbo	Observation compression	18.0%	12.5%	$1.67
Agentless [41]	Pipe	GPT-4o	Repo skeleton + embeddings	32.0%	–[†]	$0.70
AutoCodeRover [46]	OTA + AST search	GPT-4	AST APIs (SBFL optional)	19.0%	12.4%	$0.43
OpenHands [36]	OTA + CaA	Claude 3.5 Sonnet	Docker sandbox + delegation	26.0%	–[†]	$1.10
Devin [7]	OTA + Brain/DevBox	undisclosed	Persistent knowledge store	–	13.9%[‡]	–[‡]

[†]Not evaluated on Full; the authors cite compute cost. [‡]Devin is closed-source: the backbone model and per-instance cost are undisclosed, and the resolve rate is self-reported on a random 25% subset of Full. No peer-reviewed paper exists and the result has not been independently reproduced.

candidates. With a GPT-4o backbone, Agentless resolved 32.00% of SWE-bench Lite at an average cost of only $0.70 per problem, the highest reported figure among open-source systems at the time and a strong non-agentic baseline. OpenAI subsequently adopted Agentless as the scaffold for reporting GPT-4o and o1 results on SWE-bench Verified, describing it as the best-performing open-source scaffold available [26].

AutoCodeRover [46]. From NUS, AutoCodeRover takes a "software-engineering-oriented" approach, operating on program representations rather than raw files. It parses the codebase into an AST and provides the LLM with structure-aware search APIs. A stratified context retrieval process iteratively expands the agent's understanding, starting from issue keywords and refining through multiple search rounds until sufficient context is gathered. When a test suite is available, Spectrum-Based Fault Localisation (SBFL) augments the search with suspiciousness scores. In its base configuration on a GPT-4 backbone, AutoCodeRover resolved 19.0% of Lite and 12.4% of Full in a single pass—the figures reported in Table 1. Two extensions raise this without changing the backbone: enabling SBFL lifts the Lite rate to 22%, uniquely solving 7 instances no other configuration reaches, while counting the union of three sampled runs (pass@3) rather than one raises the base figures to 26.0% on Lite and 18.0% on Full. Cost ranges from $0.43 per task at pass@1 to $1.30 at pass@3.

OpenHands [36]. Formerly OpenDevin, OpenHands is an open-source platform (ICLR 2025, 32K GitHub stars, 188 contributors) built around the CodeAct agent. Each session runs inside a Docker container with a bash shell, IPython server, and Chromium browser. The platform supports multi-agent delegation: a generalist CodeAct agent can offload web browsing to a specialised BrowsingAgent. On SWE-bench Lite, CodeAct v1.8 with Claude 3.5 Sonnet achieved 26.0%. On SWE-bench Verified, a later submission reaches 60.6% with a single rollout and 66.4% when five trajectories are reranked by a dedicated critic model [28].

Devin (Cognition) [7]. Devin is a proprietary cloud-hosted agent positioned as an "autonomous AI software engineer." Its architecture separates a *Brain* (reasoning layer) from a *DevBox* (execution VM with shell, IDE, and browser). A persistent Knowledge store provides cross-session memory. Devin is the one system surveyed here with no accompanying paper: the only primary source is a launch blog post, which reports 13.86% resolved on SWE-bench. That is an order of magnitude above the 1.96% retrieval-augmented baseline, and it is the result that launched the agentic turn, but it was measured on a random 25% subset rather than the full test set, with no released trajectories or harness. The claim has never been independently reproduced. The one longitudinal external assessment, a month-long trial by Answer.AI, found Devin completed 3 of 20 tasks, with 14 outright failures [3]. We include Devin because of its architectural influence and its role in the benchmark's history, not as a comparable data point; its row in Table 1 should be read as a vendor claim rather than a measurement.

Taken together, these five systems expose a recurring trade-off between agentic flexibility and pipeline efficiency. Agentless shows that a fixed localise–repair–validate pipeline can outperform interactive agents on SWE-bench Lite at lower cost, challenging the assumption that autonomy is necessary for strong results. This claim needs care, because the rows of

Table 1 do not share a backbone: Agentless runs on GPT-4o and SWE-agent's published figure on GPT-4 Turbo, and §3.3 shows that swapping the backbone alone can move a resolve rate by ten points. The claim survives only because Agentless's authors also report the matched comparison, running the agentic baselines on the same GPT-4o backbone, and the gap persists there [41]. It is that matched comparison, not the cross-backbone table, that carries the argument. Among agentic designs, interface and context strategy matter as much as the backbone LLM: SWE-agent's ACI ablations, OpenHands's CodeAct action space, and AutoCodeRover's AST search APIs each demonstrate that how an agent reads and edits code can shift resolve rates by several percentage points at comparable model capacity.

3.3 Full-Pipeline Evaluation

The *Evaluation* branch of Figure 2 summarises the benchmark splits, scoring protocols, and confounds that shape current performance reports.

SWE-bench. The primary evaluation instrument for end-to-end agents is SWE-bench [19]: 2,294 merged pull requests scraped from 12 Python repositories, retaining only PRs that both resolve a linked GitHub issue and modify the project's test suite. The two filters do different work. The test-suite filter supplies the grading oracle; the issue-linkage filter supplies the task's *intent*, and it is the reason an agent has any natural-language input at all. An agent receives the repository snapshot from *before* the merge together with the issue text; its patch is applied and scored against *fail-to-pass* tests (which must flip) and *pass-to-pass* tests (which must not regress). The headline metric throughout this report is the *resolve rate*: the percentage of instances for which both test sets pass on the first submitted patch (pass@1).

Terminology and comparability. The name "SWE-bench" denotes a family of task sets rather than a single benchmark, and its splits differ in construction as much as in size. Table 2 summarises the vocabulary adopted throughout this report. Two distinctions warrant emphasis. First, Verified is the most carefully curated split rather than the most demanding one: human screening removed underspecified issues and unfair tests, so resolve rates on Verified are systematically higher than on Full or Lite. Scores are consequently not comparable across splits, and the contemporary frontier is measured more informatively on Pro, which is both harder and constructed to resist memorisation. Second, the labels *easy*, *medium*, and *hard* do not designate splits. They are annotator difficulty tiers defined within Verified according to the time a human engineer was estimated to require (<15 min, 15 min–1 h, >1 h) [16].

Table 2 The SWE-bench family. Lite and Verified are subsets of Full and share its 12 Python repositories; Multilingual and Pro are independently constructed task sets that reuse the harness. Resolve rates are *not* comparable across rows.

Split	Size	Selection criterion
Full	2,294	All scraped PRs that touch tests
Lite	300	Single-file patches, self-contained issues; some issue texts leak the solution [41]
Verified	500	Human-screened for issue clarity and test fairness [26]
Multilingual	300	Same pipeline, 9 non-Python languages [31]
Pro	1,865	Long-horizon multi-file changes; copyleft and held-out proprietary repos [9]

Task complexity profile. The retained tasks sit at the small end of real-world software engineering [16, 11]. Annotators judged roughly nine in ten Verified instances fixable in under an hour, and even the hard tier averages well under a hundred changed lines. Instances are overwhelmingly single-file bug fixes rather than feature work or refactorings, and heavily concentrated in a few repositories, with Django alone contributing close to half. SWE-bench therefore measures localised repair on mature Python codebases rather than software engineering in the broad sense, a caveat that shapes how its leaderboard should be read.

Leaderboard trajectory. At launch, retrieval-augmented prompting of a frontier model scored barely above zero [19]: the models could write plausible patches but had no way to find the right file or check their own work. The first agentic scaffolds (SWE-agent, AutoCodeRover, Devin) moved the frontier by an order of magnitude within months without any change to the underlying model, purely by supplying tools, a repository to navigate, and tests to run against. The 2024 scaffolding race that followed exposed how much of that gain came from scaffolding rather than reasoning capacity, since a deliberately

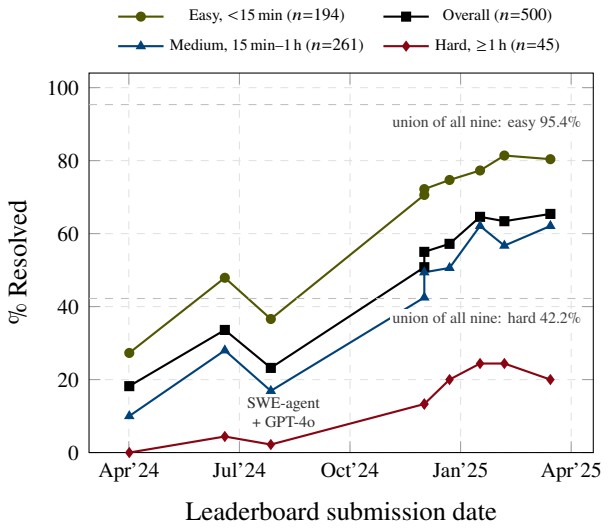

Figure 4 SWE-bench Verified resolve rate over time, overall and split by annotator-estimated difficulty, for the nine systems whose per-tier results have been published. Resolve rates and difficulty tiers come from a single re-analysis of the leaderboard's predictions [16], using the human annotations shipped with Verified [26], so the four series are directly comparable. Each point is placed at that submission's date in the leaderboard's result archive [32]. The curves connect the nine systems in submission order; they are *not* a running best-in-class frontier, which is why they dip in July 2024, when SWE-agent was resubmitted with a weaker backbone than the Claude 3.5 Sonnet run of the previous month. Dashed lines mark the *union* over all nine systems: the instances solved by at least one of them. The overall series is the tier-weighted average of the other three, with one exception: for SWE-agent + Claude 3 Opus the source's headline 18.2% (91/500) exceeds the sum of its own per-tier counts (79/500).

simple localise-then-repair pipeline such as Agentless stayed competitive with far more elaborate agentic loops.

A headline resolve rate hides where that progress actually landed. Figure 4 therefore splits Verified by the difficulty its human annotators assigned, tracking nine leaderboard systems from April 2024 to March 2025. Three things stand out, and none of them is visible in the overall curve. First, the aggregate score is dominated by composition. Easy and medium tasks are 91% of Verified, so the headline number mostly reports how well an agent handles work a developer would finish within an hour. Second, the tiers improved at very different rates. Across the nine systems the easy tier rose from 27.3% to 80.4%, a factor of 2.9, while the medium tier rose from 10.0% to 62.1%, a factor of 6.2. The easy tier is now near its ceiling, so almost all recent movement in the overall score is coming from the medium tier. Third, the hard tier behaves differently again: it climbed from 0% to about 20% by the end of 2024 and then stopped, with the last three systems scoring 24.4%, 24.4% and 20.0%. A system can gain ten points overall while solving no additional hard task, which is what makes the overall number a poor proxy for progress on the tasks that matter most.

The union sharpens this. Pooled across all nine systems, some agent solves 95.4% of easy and 84.3% of medium instances but only 42.2% of hard ones, and just 9 of the 25 hard multi-file issues [16]. The remaining easy and medium headroom is therefore largely an ensembling problem: different scaffolds already solve different instances, which is why the inference-time scaling discussed in §6 pays off. The hard tier is not an ensembling problem. More than half of it defeats every system at once, which points to a capability ceiling rather than a sampling one.

Two caveats bound how far the figure can be read. First, it plots nine independent submissions in date order, not a running best-in-class frontier, and it stops in March 2025 because the re-analysis behind it does. The July 2024 dip shows why the distinction matters: the same SWE-agent scaffold resolves 33.6% with a Claude 3.5 Sonnet backbone in June and 23.2% with GPT-4o six weeks later. Scaffolding is not the only thing that moves these curves. Lite and Full, meanwhile, receive few submissions at all now that teams have migrated to Verified [33], and the best rates ever recorded on them, 60.3% and 52.6% [32], reflect that thinning field as much as the difficulty of the splits themselves. Second, contamination sits on top of everything: half the issues predate 2020, in repositories that are staple training data [11]. Frontier models score tens of points lower on Pro, whose problems are novel and built to resist memorisation, than on Verified; §6.2 quantifies that gap and asks how much of it can honestly be attributed to memorisation.

Beyond SWE-bench. Pro and Multilingual (Table 2) answer the contamination and language critiques from within the SWE-bench format. A separate line of work leaves the format altogether and scores agents by the compute they spend rather than by resolve rate alone [15]. §6.2 treats the evaluation landscape in full.

4 Planning and Search Strategies for Code Synthesis

Planning is the stage in which an agent turns a vague intent into an operational hypothesis about *what* code should change and *how* the change can be produced.

In the vibe coding pipeline, this stage sits between intent capture and code generation, but in practice it is rarely a single up-front step. Agents plan while reading issue reports, searching repositories, choosing files, decomposing edits, sampling candidate patches, and revising their strategy after failed tests. Thus, planning is best understood as a control layer over generation rather than as a separate document written before generation begins. While §3 evaluates end-to-end agents primarily on SWE-bench, the planning and search mechanisms below are studied across function-level benchmarks (HumanEval, MBPP), competitive programming (CodeContests), and repository repair tasks alike. This section instantiates the *Planning & Search* branch of Figure 2.

This section surveys four aspects of that control layer:

1. Intent capture: how the natural-language task enters the pipeline, and what happens when it is underspecified;
2. Plan representations: determine what intermediate state the agent exposes to itself: natural-language rationales, hierarchical task lists, program sketches, or repository-localisation hypotheses;
3. Search strategies: determine how many possible solutions are explored and how candidates are ranked;
4. The plan-generate interface: determines whether planning happens once, repeatedly, or only when execution feedback invalidates the current hypothesis.

Table 3 Planning and search mechanisms in LLM-based code synthesis.

Mechanism	Planning object	Role in code tasks
Chain / tree / graph of thoughts	Intermediate reasoning steps	Explore alternative designs, algorithms, or debugging hypotheses before emitting code
Hierarchical decomposition	Subtasks, files, functions, edit locations	Reduce repository-scale tasks into bounded local edits
Candidate sampling	Multiple complete or partial programs	Trade inference cost for higher pass@k and diversity
Execution-guided selection	Tests, generated tests, compiler output	Rank or prune candidates using external evidence rather than model confidence alone
ReAct-style replanning	Updated belief state after tool observations	Interleave search, reading, editing, and feedback in a single loop

4.1 Intent Capture: The Open-Loop Stage

Intent enters the pipeline as a GitHub issue, a bug report, or a sentence typed into a chat box. It is the only stage of Figure 1 that no surveyed system instruments, and the asymmetry is worth dwelling on, because it is not obviously the right design.

Intent quality dominates outcomes. The clearest evidence comes from the benchmarks themselves. SWE-bench Verified was built by having human engineers discard instances whose issue text was *underspecified*, that is, instances where the intent did not determine the patch [26]. Screening the intent, while changing neither the repositories, the agents, nor the grading harness, raises resolve rates by tens of points (§3.3). The same lever appears with the opposite sign in Lite, where some issue descriptions leak the fix outright, so that the intent contains its own solution [41]. Between these two poles lies the entire difficulty range that Figure 4 stratifies. An agent's score is, to a degree that is rarely acknowledged, a score on the issue-writing of the repository it was sampled from.

No agent closes the loop here. Every other stage of the pipeline regulates on an external signal. Planning is checked against repository structure (does `search_class` find the symbol?); generation against a parser and a linter; execution against a test runner; refinement against the resulting failures. Intent capture has no such signal. None of the five systems in §3.2 asks a clarifying question when the issue text is ambiguous; each instead commits to a reading of it and proceeds. OpenHands and Devin admit human interjection (§5.5), but the initiative is the human's: the agent does not detect underspecification and request disambiguation. In control terms the stage is *open-loop*. Where the pipeline everywhere else substitutes an external oracle for the model's own confidence, at the intent stage the model's reading of the issue simply *is* the specification, unchallenged.

Why the omission persists. The benchmark makes the omission invisible. A dataset of issues that were, by construction, resolved by a real merged PR is a dataset in which the intent was adequate; and Verified deliberately removed the instances where it was not. An agent is therefore never rewarded for asking, and never penalised for guessing wrong about a genuinely ambiguous request. The cost surfaces only outside the benchmark, as the specification-ambiguity challenge

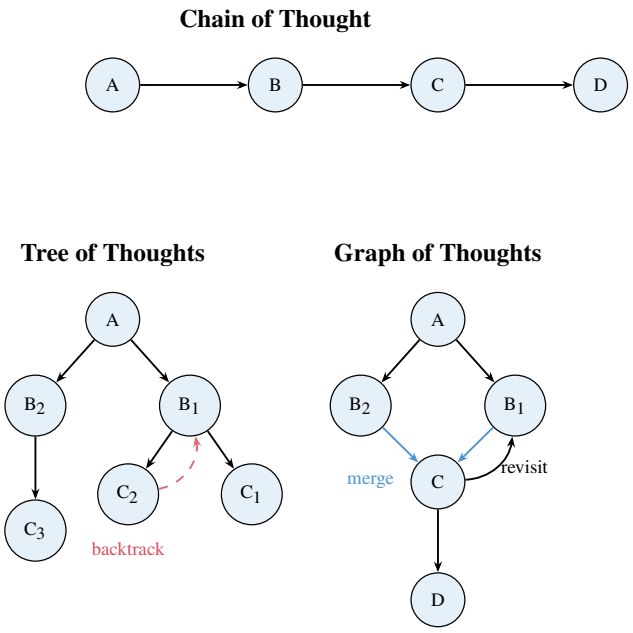

Figure 5 Plan representation structures. Chain of Thought commits to a single linear path. Tree of Thoughts explores alternatives and backtracks. Graph of Thoughts merges branches and revisits earlier states.

of §6.4. We regard this as the clearest structural gap the pipeline framing exposes: four stages have been given feedback, and the fifth has been given a leaderboard that hides the fact that it has none.

4.2 Plan Representations and Decomposition

Plan representations determine how an agent organises reasoning before generating code. The simplest is chain of thought [39]: a linear sequence that commits early to one trajectory. If the first design choice is wrong, subsequent steps elaborate the mistake. Tree of Thoughts [44] and Graph of Thoughts [4] relax this by allowing exploration, backtracking, and merging across alternative paths (Figure 5). The mechanism in Tree of Thoughts is worth naming precisely, since the figure's "backtrack" arrow understates it: the tree is expanded under a classical search policy (breadth- or depth-first) in which an LLM *value function* scores each partial state, so branches are pruned by lookahead rather than abandoned only once they fail. The evaluator, not the branching, is what makes the structure useful, and it is also what a repository-scale agent lacks, since no cheap value function scores a half-finished patch. For code synthesis, this means agents can consider multiple decomposition strategies ("change the parser", "change the caller", or "adjust the compatibility layer") as explicit search states rather than implicit in a single sample.

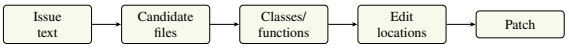

Figure 6 Hierarchical localisation pipeline. Agents narrow scope iteratively from repository-level to line-level edits.

Repository-level agents usually add a more concrete representation (recall §3.2): *localisation plans*. Agentless decomposes issue resolution into files, classes/functions, and edit locations before generating a patch [41]. AutoCodeRover uses AST-backed APIs such as `search_class` and `search_method_in_class` to plan over program structure rather than raw text [46]. These grounded representations name concrete program elements that can be validated against the repository (Figure 6), turning a broad intent into a bounded search problem.

Plans may also be expressed as *specification sketches*: partial interfaces, pseudocode, assertions, or examples that constrain the generated implementation. Useful plans are grounded in observable artefacts (issue text, existing code, API conventions, and failing tests) rather than invented requirements.

4.3 Search and Exploration Strategies

Once a representation exists, an agent must decide how broadly to search. Single-sample generation is fast, but it treats the model's first completion as the solution. Search-based systems instead spend additional inference or execution budget to explore alternatives. The simplest form is self-consistency: sample multiple reasoning paths and select the most common or most consistent answer [37]. Transposing this to code forces the idea to mutate, and the reason is instructive. Self-consistency marginalises over reasoning paths that terminate in a *single canonical answer*, so votes can be counted by string equality. Programs have no canonical form: two correct patches may share no token, and two textually near-identical patches may differ semantically. The vote must therefore be taken in a different space. Systems recover it either *behaviourally*, by clustering candidates on their outputs over a set of inputs (AlphaCode, CodeT), or *syntactically up to normalisation*, by comparing candidates after AST canonicalisation (Agentless [41]). Sample-and-rank is thus self-consistency with an equivalence relation borrowed from program semantics rather than from string matching, and the quality of that relation bounds what the vote can buy.

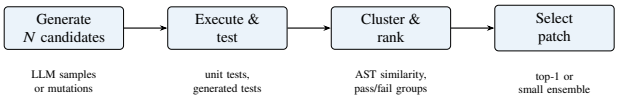

Figure 7 Candidate-selection pipeline. Systems like AlphaCode and CodeT generate many candidates, execute them against tests (real or generated), cluster by behaviour, and select a small high-quality set.

AlphaCode [22] and CodeT [5] exemplify this pattern (Figure 7). AlphaCode generated millions of candidates for competitive programming, filtered and clustered by execution results, then submitted a small diverse set. CodeT generates both programs and tests, selecting code consistent with generated test cases. The key insight is that LLMs often contain the right solution somewhere in the sample distribution even when top-ranked output is wrong. The limitation is self-referential objectives: if generated tests are weak, selection optimises for the model's own misunderstanding. Execution-guided search is strongest when feedback includes external oracles (existing unit tests, compiler errors, type checkers, or repository-specific regression suites).

Search also appears in repository navigation, as in the systems surveyed in §3. Agentless samples multiple locations and candidate patches, then normalises patches by AST and uses validation to pick among them [41]. AutoCodeRover expands context iteratively through program-structure queries, optionally using Spectrum-Based Fault Localisation (SBFL) to prioritise suspicious code when tests are available [46]. In both cases, search is not over complete programs alone; it is also over *where to look* and *what evidence to include*. This matters because code generation quality is bounded by context quality. A correct patch is unlikely if the agent never places the relevant function, invariant, or failing trace in context.

The cost of search is the main trade-off. More samples increase diversity and pass@k, but they also increase latency, token cost, and validation cost. Search can also overfit to narrow tests: a candidate that passes the available suite may still violate unstated requirements. Strong systems therefore combine multiple pruning signals: syntactic validity, static analysis, failing-test repair, pass-to-pass regression preservation, and semantic similarity to the intended behaviour. Search is most useful when the ranking signal is at least as reliable as the generator is diverse.

Table 4 Plan–generate coupling patterns.

Pattern	Mechanism	Trade-off
Plan-then-generate	Plan once, then generate code conditioned on it	Easy to inspect; brittle if plan is wrong
Interleaved planning	Alternate plan updates and tool use in observe–think–act loop	Adaptive; plan distributed across turns
Generate-then-refine	Generate, critique via feedback, revise iteratively	Self-correcting; may reopen planning after execution

4.4 Plan–Generate Interplay

Planning and generation can be coupled in three broad ways (Table 4). *Plan-then-generate* is easy to inspect and aligns with human practice, but brittle if the initial plan is wrong [41]. *Interleaved planning* adapts to new information but distributes the plan across many turns, making it harder to audit [45]. *Generate-then-refine* treats the first output as a hypothesis and revises via self-critique or execution feedback [23], connecting planning to the refinement loop in §5.

The central design question is not whether agents should plan, but how plans remain *grounded*. Grounding mechanisms include retrieval over repository structure, AST-level symbol search, execution traces, failing tests, and explicit constraints extracted from the issue report. Effective systems keep plans close to these artefacts and treat generation as one move in a larger search process, reframing code synthesis from "write the answer" to "maintain and test a hypothesis about the required program change".

This distinction resolves what would otherwise be an embarrassment for the control-layer thesis: Agentless, the system that plans *least*, is competitive with those that plan most (§3.2). If the claim were that deliberation helps, Agentless would refute it. The claim is instead that *grounding* helps, and deliberation is merely one way to obtain it, often an expensive one. Agentless obtains grounding structurally, by hard-coding the hierarchical narrowing that an agentic loop must rediscover through exploration, and it therefore confirms the thesis rather than embarrassing it. The corollary is that finding the right location is harder than editing it once found: AutoCodeRover's SBFL variant changes nothing about repair, only about localisation, and buys three points of resolve rate together with seven instances no other configuration reaches [46]. Planning

in this setting is not reasoning about *how* to write code. It is reasoning about *where* the code is, and about what evidence would confirm that guess.

5 Execution Feedback and Iterative Refinement

Building on the plan–generate patterns surveyed in §4, this section examines how execution closes the loop. Code generation rarely succeeds in one shot. Even human programmers iterate: write, compile, test, debug, revise. Agentic systems execute generated code, observe runtime outcomes (test failures, compiler errors, exceptions), and revise their output accordingly. While §3 scores agents on SWE-bench, refinement mechanisms are often evaluated on function-level suites (HumanEval, MBPP), text-to-SQL (Spider), and APR benchmarks (Defects4J). This section instantiates the *Execution Feedback* branch of Figure 2 and examines the feedback signals agents consume (§5.1), the mechanisms by which they refine code (§5.2), the evidence on whether refinement actually works (§5.3), the techniques for localising and repairing faults (§5.4), and the criteria for deciding when to stop iterating (§5.5).

One distinction organises the whole section, and we state it up front rather than deriving it late: *what closes the loop*. A refinement mechanism is only a control loop if some signal outside the model's own judgement can contradict the model. Where that signal is an external oracle (a test runner, a compiler, a type checker) refinement reliably improves code. Where the signal is the model's own critique of its own output, the loop is closed onto itself and the apparent control is illusory. Table 5 sorts the mechanisms surveyed below on exactly this axis, and §5.3 supplies the evidence that the axis is the one that matters.

5.1 Feedback Signals and Environments

Agents require feedback to improve. The type, source, and presentation of feedback critically affect refinement efficacy.

Types of feedback. The most common feedback is *test execution results*: pass/fail verdicts, failing test cases, assertion messages, and expected-versus-actual output diffs. When tests fail, agents receive concrete evidence of incorrectness. A richer signal is *compiler or interpreter errors*: syntax errors, type mismatches, undefined-variable messages, and stack traces. These pinpoint the location and nature of faults without requiring test design. Static analysis tools provide another layer: linters flag style violations, type checkers identify inconsistencies, and data-flow analysers detect null dereferences or race conditions before execution. Some systems incorporate *human feedback* in the loop: developers approve or reject patches, provide natural-language explanations of failures, or annotate bug reports. Finally, agents can generate *self-critique*: an LLM explains the generated code in natural language, identifies logical flaws, or predicts potential failures without executing the code ("rubber duck debugging") [6].

Feedback sources: external versus internal. Feedback may come from *external* oracles (pre-written test suites, continuous-integration pipelines, runtime environments) or be *internally simulated* by the agent itself. The distinction is not a taxonomy convenience; it is the load-bearing variable of this section. External feedback is authoritative because it can *surprise* the model: a test that fails contradicts the model's belief regardless of how confident that belief was. It requires infrastructure and may be slow or expensive. Internal feedback is fast and requires no setup, but it is generated by the same distribution that produced the error, so it inherits the error's blind spots: the agent may confidently believe its code is correct, or fabricate an error message for a line that is fine. An agent that could reliably tell its wrong code from its right code by inspection would have produced the right code in the first place.

Table 5 sorts the mechanisms of §5.2–§5.4 by what closes their loop. Hybrid approaches, which use external execution for validation and internal self-critique for *diagnosis*, occupy the productive middle: the oracle decides *whether* the code is wrong, the model proposes *why*. §5.3 argues that this division of labour, and not self-critique as such, is what makes refinement work.

Sandboxed execution environments. To safely run untrusted generated code, systems employ sandboxing. Common techniques include *containerisation* (Docker), *virtual machines*, *language-level isolation* (restricted Python interpreters), and *time-out and resource limits*. SWE-agent [43] and AutoCodeRover [46] execute each task inside a per-instance Docker container, which is what makes SWE-bench's environment-dependent test suites reproducible at all; OpenHands additionally runs a

Table 5 Refinement mechanisms sorted by what closes the loop. Only an external oracle can contradict the model; internal signals can only reorganise what the model already believes.

Mechanism	Loop closed by	Signal	Characteristic failure
Self-Refine [23]	Internal	Model critiques own output	Plateaus; no signal can contradict the generator
Self-debugging, no tests [6]	Internal	Self-generated line-by-line explanation	Explains the bug as intended behaviour
Self-debugging, with tests [6]	Hybrid	Test verdict + model diagnosis	Bounded by test coverage
Reflexion [30]	Hybrid	External reward/verdict, verbalised into memory	Reward must exist and be dense enough
SelfEvolve [18]	External	Interpreter errors, traces	Silent (non-crashing) semantic bugs
D4C [42]	External	Failing tests + full function	Plausible-but-incorrect patches
Agentless validate [41]	External	Regression + generated tests	Generated tests inherit the model's misreading
Best-of-N + critic [28]	Hybrid	Executed rollouts ranked by learned critic	Cost scales with N

browser and an IPython server in the same sandbox [36]. Sandboxing serves two purposes that are easily conflated: *reproducibility*, ensuring the harness scores the patch and not the environment, and *containment*, preventing generated code from damaging the host. Most research systems are engineered for the former; the latter matters once agents run against real infrastructure (§6.4). Both introduce latency and restrict access to the databases and network services that production code needs, so balancing safety and realism remains an open design challenge.

5.2 Iterative Refinement Mechanisms

Once feedback is available, agents must incorporate it to produce better code. Refinement strategies vary in how they represent feedback, how they prompt the model to revise, and whether they maintain memory across iterations.

Reflexion: verbal reinforcement learning. Reflexion [30], from Northeastern, MIT, and Princeton, treats refinement as a form of reinforcement learning without weight updates. After each trial, the agent reflects on what went wrong by generating a natural-language critique ("The function failed because it did not handle negative inputs"). This reflection is stored in an episodic memory buffer and retrieved in subsequent attempts, conditioning the model to avoid past mistakes. Reflexion accepts diverse feedback: scalar rewards (e.g., pass rate), binary verdicts, or free-form error messages. The key innovation is converting feedback into *verbal* form that persists across trials, enabling the model to learn from experience within a single episode. On HumanEval, Reflexion achieves 91% pass@1, surpassing single-shot GPT-4 (80%) by explicitly encoding trial-and-error history. The approach generalises beyond code: it improves sequential decision-making in games and language reasoning tasks.

Self-debugging: rubber duck debugging. Chen et al. [6] propose *self-debugging*, where the model critiques and repairs its own code without external feedback. The agent generates code, then *explains* the code line-by-line in natural language. This explanation serves as a self-generated specification, allowing the model to spot discrepancies between intent and implementation. If the explanation reveals a flaw ("This loop iterates over indices, but I wanted to iterate over values"), the model revises the code. Self-debugging also incorporates external execution feedback when available: the model receives error messages or failing test outputs and reasons about root causes before proposing fixes. On Spider (text-to-SQL), self-debugging improves baseline accuracy by 2–3% without unit tests and by up to 12% on MBPP when tests are available. The method demonstrates that internal self-critique complements external execution signals.

Repair prompts and diff-based editing. Simpler refinement strategies use *repair prompts*: the agent receives the original task, the buggy code, and the error message, then generates a revised version. To reduce token costs, some systems use *diff-based editing*, where the model outputs only the changes (insertions, deletions) rather than the full code. This is particularly effective for large files: instead of regenerating hundreds of lines, the agent specifies "replace line 42" or "insert after line 57". SWE-agent's edit tool and Agentless's diff-based repair both adopt this paradigm. The trade-off is that models must learn the diff format, and parsing errors can break the repair loop.

5.3 Does Refinement Work? The Limits of Self-Correction

The results above are usually reported as evidence that agents improve their own code. Two lines of work complicate that reading, and taking them seriously sharpens rather than weakens the control-layer thesis.

Self-repair is partly inference-time scaling in disguise. Olausson et al. [25] observe that a self-repair loop does not spend a fixed budget: producing a critique and a revision consumes tokens and samples that could instead have been spent drawing more independent candidates from the generator. When they normalise for that budget, comparing self-repair against simply sampling k programs i.i.d. and taking the best, much of self-repair's apparent gain disappears. The gain that survives is attributable to the *feedback*, not to the iteration: repair helps most when the model receives a stronger critique than it can produce for itself, and in their strongest condition the critique comes from a human. Reflexion's HumanEval headline (§5.2) should therefore be read as a result about a *budget*, not only about a mechanism, and compared against a best-of-N baseline at matched cost rather than against single-shot GPT-4.

Intrinsic self-correction does not reliably help. Huang et al. [17] isolate the case where no external signal is available, which they term *intrinsic* self-correction, and find that models asked to review and revise their own reasoning often make it worse, converting correct answers into incorrect ones roughly as often as the reverse. The mechanism is unsurprising given §5.1: the critic and the generator are the same distribution, so the critique cannot introduce information the generator lacked. Reported gains from "self-critique" commonly depend on an oracle that has quietly leaked into the loop, most often a stopping rule that halts only once the answer is right.

What this implies for the pipeline. The two results converge on one conclusion, which we take as this report's sharpest claim about refinement. *Refinement is a control layer only when an external oracle closes the loop; internal self-critique is an open loop wearing a control loop's clothes.* The framing explains the pattern in Table 5: every mechanism that survives contact with repository-scale evaluation (Agentless's validation phase, SWE-agent's edit–execute loop, D4C's test-driven repair) is externally closed, and the purely internal mechanisms cluster in function-level benchmarks where a comprehensive test suite is nonetheless available to the *evaluator*, if not to the agent.

It also unifies two techniques that the literature treats separately. Iterative self-repair and best-of-N sampling with a reranker are the same lever, a budget of extra inference spent to reduce variance, pulled in two different directions: sequentially, conditioning each sample on the last, or in parallel, conditioning none of them. Once budget is held fixed the interesting question is not "does the agent refine?" but "which allocation of the same tokens buys more correctness?" OpenHands's Verified result answers it in one direction: five parallel rollouts reranked by a dedicated critic reach 66.4% where a single rollout reaches 60.6% [28]. Notably, the critic is a *separate, trained* model rather than the generator judging itself, which is precisely the condition Olausson et al. identify as the one under which feedback helps. The cost of this lever, and its consequences, return in §6.4.

5.4 Fault Localisation and Self-Repair

When refinement targets bugs in existing code rather than improving draft solutions, it becomes *automated program repair* (APR). The classic APR pipeline has three stages: fault localisation (identify buggy lines), patch generation (propose fixes), and patch validation (test the fix). LLM-based agents challenge this structure by generating end-to-end repairs.

Traditional fault localisation. Pre-LLM APR systems rely on *spectrum-based fault localisation* (SBFL), which ranks code lines by suspiciousness scores derived from test coverage: lines executed by failing tests and not by passing tests receive high scores. Ochiai, Tarantula, and DStar are standard SBFL metrics. LLM-based systems initially adopted this pattern, using SBFL to extract a small code snippet and prompting the model to repair it. However, fault localisation is imperfect: even oracle-provided bug locations ("perfect FL") do not guarantee correct repairs, and SBFL scores are often noisy.

FL-free repair with objective alignment. Recent work questions whether explicit fault localisation is necessary. D4C [42] argues that LLMs should repair entire functions rather than isolated hunks, aligning the task with their pre-training objective (next-token or infilling completion). D4C prompts the model with the full function, failing test cases, and error messages, and

asks it to output a corrected function. On Defects4J, D4C repairs 180 single-function bugs with only 10 samples per bug, outperforming systems with perfect fault localisation by 10% and reducing sample count by 90%. The insight is that LLMs, trained on vast corpora of correct code, implicitly learn to locate faults when given sufficient context; forcing them to target specific lines may constrain their reasoning.

Reconciling FL-free repair with localisation-first agents. Read beside §4, D4C appears to contradict the systems surveyed there. Agentless and AutoCodeRover treat localisation as the core of the problem and spend most of their pipeline on it; D4C reports that localisation is unnecessary and even harmful. Both are right, because they operate in different regimes, and naming the regime is more useful than picking a winner.

D4C's Defects4J tasks are *single-function* bugs: the buggy function is supplied. What D4C eliminates is therefore not the search for the faulty function but the finer-grained ranking of *lines within* a function already known to be faulty, and its argument, that line-level targeting fights the model's infilling prior, is convincing at that granularity. A SWE-bench agent faces a prior question, which function among tens of thousands, and no pre-training prior answers it: the relevant symbol may not appear in the issue text at all. The reconciliation is a claim about scale. *Below* the function boundary, localisation is a constraint the model does not need and should not be given; *above* it, localisation is the task. This is why AutoCodeRover's SBFL variant helps (it ranks functions) while D4C's ablation of perfect FL helps (it un-ranks lines), and the two results, superficially opposed, are measuring opposite sides of the same boundary.

Stack-trace analysis and root-cause reasoning. When execution fails, agents receive stack traces showing the call chain and exception type. Stack traces are rich signals: they identify the failing function, reveal the exception (NullPointerException, IndexError), and often include variable values. Agents can parse stack traces to infer root causes: "IndexError at line 15 suggests the list is shorter than expected; check loop bounds." SelfEvolve [18] and SWE-agent [43] iteratively refine code by feeding error messages back to the LLM, which reasons about the fault and proposes targeted edits. The challenge is that stack traces can be noisy (multiple exceptions, deep call stacks) and may not reveal semantic bugs (e.g., off-by-one errors that do not crash).

Test-driven repair loops. A refinement loop driven by test feedback proceeds as follows: (1) generate initial code, (2) run tests, (3) if any fail, extract failure messages, (4) prompt the model to fix the code, (5) repeat until all tests pass or a budget is exhausted. The stopping condition is critical: systems may iterate 1, 3, 5, or 10 times. Empirically, most gains occur in the first 2–3 iterations; further iterations yield diminishing returns and risk introducing new bugs. SWE-agent's edit–execute loop and Agentless's repair–validate pipeline both iterate until tests pass or a budget is exhausted. The iteration budget balances quality and cost.

Candidate ranking and ensemble repair. When multiple repair attempts produce different patches, agents must select the best one. Ranking strategies include: (i) test coverage (choose the patch that passes the most tests), (ii) LLM-as-judge (ask the model to explain and rank patches by confidence), (iii) ensemble voting (generate N patches and pick the most common or most syntactically similar to the original), and (iv) semantic similarity (prefer patches that minimally alter the original code). CodeT [5] and Agentless [41] both rank candidates by test execution, filtering implausible patches before final selection.

5.5 Convergence and Stopping Criteria

Iterative refinement must terminate. Agents need stopping criteria that balance solution quality, cost, and the risk of overfitting to weak test suites.

Pass-rate thresholds. The simplest criterion is *full test pass*: stop when all provided tests pass. This works well for competitive programming (HumanEval, CodeContests) where test suites are comprehensive. However, in software engineering tasks (SWE-bench), tests may be incomplete or incorrect. A patch that passes all tests may still be wrong (a "plausible but incorrect" patch), and a correct patch may fail tests due to environment issues. Note that the criterion is also, quietly, an oracle: as §5.3 observed, a loop that halts exactly when the answer becomes right has smuggled ground truth into its stopping rule, and any self-critique operating inside such a loop will appear more effective than it is.

Diminishing improvement detection. If successive iterations yield no improvement (measured by test pass rate, execution success, or the failing test remaining unchanged), the agent should stop. If iterations 3, 4, and 5 all produce code that fails the same test with the same error, further iteration is unlikely to help, and the heuristic prevents endless loops in which the model oscillates between two incorrect states. The empirical basis for stopping early is the shape of the repair curve: Olausson et al. find that the marginal return of an additional repair round falls off quickly and, once the token budget is held fixed, is frequently negative relative to drawing a fresh independent sample [25]. Where the loop is closed only internally, Huang et al. find the marginal return can be negative in absolute terms [17]: iterating past the first round makes the answer worse. The practical implication is that a stopping rule and a budget allocation are the same decision, and that "iterate until it passes" is defensible only when an external oracle defines *passes*.

Budget limits: iterations and tokens. Practical systems impose hard limits. Iteration budgets (3, 5, 10 rounds) cap the number of refinement cycles. Token budgets limit total inference cost: after consuming a threshold, the agent returns its best attempt. Reflexion and Agentless both use iteration caps, while D4C's low sample count (10 tries per bug) implicitly limits token use. The optimal budget depends on task difficulty: simple bugs may resolve in one iteration, while complex logical errors may require many. The budget is not merely a cost control, however. Because sequential refinement and parallel best-of-N draw on the same pool of tokens (§5.3), fixing the budget is what makes the two strategies comparable at all, and an iteration cap is implicitly a bet about which of them converts tokens into correctness more efficiently for the task at hand.

Risk of overfitting to weak test suites. A critical failure mode is overfitting. If the test suite is weak (few tests, shallow coverage), the agent may generate code that narrowly passes those tests but fails on unseen inputs. This is the "plausible patch" problem in APR: patches that satisfy the validation oracle but do not generalise. Mitigations include: (i) *test augmentation*, where the agent or a separate test generator produces additional test cases to catch edge cases; (ii) *static checks*, requiring generated code to satisfy linting, type checking, or invariant assertions beyond test pass; (iii) *semantic similarity constraints*, penalising patches that radically alter program structure; and (iv) *LLM-based verification*, asking the model to critique the patch or predict whether it will generalise. However, no technique fully eliminates overfitting, and the problem remains open.

Human-in-the-loop approval. In high-stakes domains (medical software, financial systems), automated iteration may be unacceptable without human oversight. Systems like OpenHands and Devin allow developers to inspect intermediate outputs, approve or reject patches, and provide guidance before the next iteration. This trades speed for safety and introduces latency, but ensures that convergence criteria align with human judgment rather than proxy metrics.

6 Discussion

Having surveyed end-to-end systems (§3), planning and search (§4), and execution-feedback refinement (§5), we now consolidate what is common across these fronts and which cross-cutting constraints bound all of them. Progress has come less from any single architectural idea than from *coupling* generation to external evidence, and that same coupling now exposes the field's hardest problems: evaluation integrity, finite context, cost, and trust.

6.1 Consolidated Patterns and Trends

Three design patterns recur across nearly every system we surveyed, and one consequence follows from all three.

The *observe–think–act loop* [45] is the near-universal control structure (§3.1); Agentless's fixed pipeline [41] is the exception that stays competitive by hard-coding the sequence the loop would otherwise discover. *Execution-grounded selection*, which ranks sampled candidates by tests, generated tests, or regression preservation rather than by model confidence (AlphaCode [22], CodeT [5], Agentless), is the most reliable quality lever. And *planning functions as a control layer over generation* rather than a standalone up-front step (§4).

The unifying observation is that the second pattern explains the other two. Everything that works, works by putting something outside the model in a position to contradict it: a test runner, a compiler, an AST index, a trained critic. Planning earns its place by grounding hypotheses in repository structure that can be checked (§4); refinement earns its place only when

an external oracle closes the loop, and degrades to noise when it does not (§5.3). Read this way the pipeline is not five stages of reasoning but a scaffold for arranging confrontations between a generative model and an environment that can say no. Its two weakest points are exactly the two places where nothing can: intent capture, where no oracle exists at all (§4.1), and test-based validation, where the oracle exists but is weak enough to be gamed (§6.2).

This suggests a sharper way to state the control-layer thesis. A control loop regulates a plant against a *setpoint*; for these agents the setpoint is "the tests pass." But §5.5 and §6.4 together establish that this setpoint is a proxy, and a leaky one: patches that pass can be overfitted, insecure, or the product of reward hacking. Current agents are therefore control loops regulating tightly on a signal that only approximates what we want. Their impressive convergence and their characteristic failures are the same phenomenon viewed from either end.

Three macro trends follow. The field has moved from single-shot prompting to agentic scaffolds; from flat chain-of-thought to structured search and localisation; and, most recently, from a one-shot rollout to *inference-time scaling*, where many trajectories are sampled and reranked by a critic model, as OpenHands's 60.6% → 66.4% Verified result illustrates [28]. A consistent finding across §3 is that *scaffold and interface matter as much as the backbone model*. That finding, however, is time-stamped rather than timeless. A newer line of work moves the scaffold's competence into the weights, training agents on execution feedback directly, either by reinforcement learning over software-evolution data [40] or by training agent and verifier together in an executable environment [29]. If this succeeds, the scaffolding advantage that §3 documents is a transitional artefact of using models that were never trained to be agents.

A parallel shift has occurred in deployment: the systems surveyed here are research artefacts, while the agents developers actually use (Claude Code, Cursor, OpenAI's Codex) are products whose scaffolds are undocumented and whose evaluations are self-reported, which is one reason the benchmark integrity questions of §6.2 have become urgent.

6.2 The Evaluation Landscape

The benchmarks used to measure agentic code generation form a layered stack, and each layer is showing strain. *Function-level* suites (HumanEval, MBPP) are effectively saturated (Reflexion already reported 91% pass@1 on HumanEval in 2023 [30]) and no longer discriminate among frontier systems. *Repository-level* evaluation shifted the centre of gravity to SWE-bench [19], whose Verified split [26] has become the de-facto leaderboard standard. Yet §3.3 already noted its difficulty skew, which in numbers means 39% of tasks are trivial <15-min fixes and 86% touch a single file [16], its repository monoculture (Django dominates), and its contamination risk, since many issues predate model training cutoffs and the repositories are widely scraped [11].

Three responses have emerged. The first is *harder, contamination-resistant successors*. SWE-bench Pro [9] contributes 1,865 long-horizon problems from 41 repositories and resists memorisation using copyleft-licensed and held-out proprietary code; at release, top systems scored only ∼23% versus 70%+ on Verified, and even by mid-2026 no public system clears the high 50s [2]. SWE-bench Multilingual [31] extends the format to 300 tasks across 9 languages, addressing SWE-bench's Python monoculture. Figure 8 makes the divergence concrete: frontier models clustered near or above 80% on Verified fall to the mid-40s–high-50s on Pro, a 20–35 point gap for the *same* model weights. Claude Opus 4.5, for instance, scores 80.9% on Verified but 45.9% on Pro under Scale's standardised harness [9, 1]. Two confounds prevent reading the *size* of each model's gap as a memorisation measurement, and we flag them because they are the same confounds this report holds others to. Pro is harder as well as fresher, so difficulty and contamination are entangled in every drop; and the Pro column mixes harnesses, only Opus 4.5 being scored under Scale's standardised evaluation while the rest self-report with custom scaffolding. The systematic gap across all four models is robust to both. The per-model ranking within it is not, and we draw no conclusion from it. The signal was strong enough that OpenAI stopped reporting Verified scores in early 2026, citing near-universal contamination (models reproduce gold patches from the task ID alone) and flawed residual tests [27]. The second response targets *benchmark integrity directly*: a stricter evaluation harness that closes test-leakage channels can cut reported scores substantially (e.g., double-digit drops on SWE-bench Pro for some models), prompting the observation that "reward hacking is swamping model intelligence gains": agents partly learn to exploit weaknesses in the grading oracle rather than to solve the task [8]. The third response is *efficiency-aware evaluation*: SWE-Effi [15] re-scores agents under resource constraints and finds that resolve

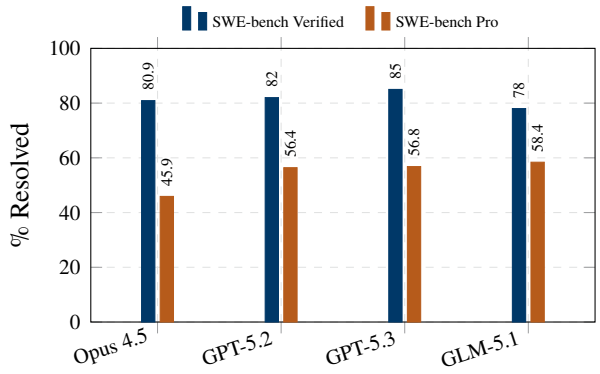

Figure 8 The Verified–Pro gap. The *same* frontier model weights score 20–35 points lower on the harder, contamination-resistant SWE-bench Pro (1,865 tasks, 41 repos, GPL+held-out code) than on the saturated, widely-scraped SWE-bench Verified (500 tasks, 12 repos). Only the *systematic* gap should be read from this figure. The per-model differences must not be: Verified scores are vendor-reported and approximate, and on Pro only Opus 4.5 is scored under Scale's standardised SEAL harness, the rest being self-reported with custom scaffolding. The model measured under the strictest harness will show the largest drop for reasons that have nothing to do with memorisation, so the spread across models here is confounded with the spread across harnesses. Data compiled April 2026 from leaderboard aggregators [1, 9].

rate decouples sharply from cost. Agentless reaches the highest resolve rate in its study (48% with Qwen3-32B) but consumes 83.1 API calls and 209.4 seconds per issue on average, so a single pass@1 number conceals an order-of-magnitude spread in cost.

A single resolve-rate number is therefore no longer a sufficient summary. Credible comparison now requires difficulty stratification, contamination controls, and cost (or token/latency) normalisation reported alongside accuracy. The pipeline framing makes the coverage gaps concrete: most benchmarks still score only the *end* of the pipeline (did the final patch pass?), leaving intermediate stages such as plan quality, localisation precision, and refinement efficiency largely unmeasured.

6.3 The Context Window Bottleneck

Every pipeline stage is constrained by the same resource: the model's finite working context. Real repositories far exceed even million-token windows, so an agent never sees the whole program and must decide *what* to read. This is not solved merely by longer context windows. Repository-level studies find that retaining the full *dependency* context yields the best results, while simply enlarging the context with loosely related code can be misleading and degrade correctness [24]: more tokens are not uniformly better. A second failure mode is dynamic: agents that append every observation to an ever-growing trajectory suffer *context explosion* and *semantic drift*, where critical early information is buried or lost as the conversation lengthens [12]. Because input tokens are re-sent each turn, trajectory length also drives cost roughly quadratically over a run [14], tightly coupling the context bottleneck to the cost challenge below.

The systems we surveyed therefore treat context as a managed resource rather than a passive log. Mitigations span the whole pipeline: hierarchical repository skeletons with embedding retrieval (Agentless [41]), AST-based structure-aware search that returns signatures instead of full bodies (AutoCodeRover [46]), and observation compression of older turns (SWE-agent [43]). The current research direction is to make context management *active*, explicitly summarising, pruning, and re-retrieving context as a deliberate agent action rather than a passive heuristic [12]. Context quality, not model capability alone, frequently determines whether the correct patch is even reachable: an agent cannot fix what it never places in context.

6.4 Open Challenges

Four challenges cut across all three fronts and remain open.

Long-horizon, multi-file reasoning. Current agents excel at small, local, single-file fixes but degrade steeply on tasks that require coordinated changes across many files and long action horizons. The easy/hard gap on SWE-bench Verified (~80% of easy vs. ~20% of hard tasks solved by the best single system [16]) and the steep drop on the explicitly long-horizon SWE-bench Pro [9] both point to the same ceiling. Sustained planning, memory, and error recovery over long trajectories remain unsolved.

Is this the same constraint as the context bottleneck above, or a distinct one? The evidence in §3.3 suggests they are distinct, and we commit to that reading. If the hard tier were context-bound, it would be an *ensembling* problem, since different scaffolds retrieve different context and would therefore fail on different instances. That is exactly what the easy and medium tiers show: pooled across nine systems, some agent solves 95.4% of easy and 84.3% of medium instances, far above any single system. The hard tier does not behave this way. More than half of it defeats all nine

systems at once, and only 9 of 25 hard multi-file issues are solved by anyone [16]. A limitation that every scaffold hits at the same instances is not a retrieval limitation. Context management is necessary and, on the evidence, insufficient: it determines whether the right patch is *reachable*, while the hard tier appears to test whether the model can construct a multi-step, cross-file change that no amount of retrieval will hand it. We would expect training-time approaches [40, 29] rather than better retrieval to move this tier, and offer that as a falsifiable prediction.

Specification ambiguity. This is the open-loop intent stage of §4.1, seen from the outside. Vibe coding starts from natural-language intent that is often underspecified. When the issue text omits edge cases or implicit invariants, the agent must guess, and execution feedback only constrains behaviour that the available tests exercise, so a wrong guess about an untested invariant is never contradicted and is scored as success. Closing the gap between an ambiguous intent and a verified specification, whether through clarification, test synthesis, or interactive intent capture, is largely unaddressed in current end-to-end systems, and current benchmarks cannot reward addressing it: a dataset built from issues that a real pull request resolved is a dataset from which ambiguity has been filtered out in advance.

Cost and efficiency. The dominant quality lever, inference-time scaling with many sampled trajectories and critic reranking, directly multiplies cost, while trajectory length inflates token usage per turn [14]. Because resolve rate decouples from compute [15], the practical frontier is not peak accuracy but accuracy per dollar and per minute. Turn-control and trajectory-reduction strategies are early attempts to make agents economically deployable rather than merely capable.

Trustworthiness and security. Passing the available tests does not make generated code safe. A large-scale evaluation across 100+ models found that 45% of generation tasks introduced a known security flaw, and that this rate has not fallen as models have grown larger and newer [34]. Purpose-built benchmarks for agentic settings reach the same conclusion: SusVibes shows that vibe-coding agents produce vulnerable implementations on realistic feature requests [13], and systematic analyses document insecure behaviours that functional test suites never catch [21]. Because refinement loops optimise for *functional* pass rates (§5.5), they do not penalise insecure-but-passing patches, a blind spot that compounds with the overfitting risk of weak test suites.

This is where the terminology of §2.1 comes due. In Karpathy's narrow sense, vibe coding is defined by the developer not reading the diff. We argued that the pipeline must then re-supply every guarantee the reader used to supply. It does re-supply some: localisation replaces the developer's knowledge of where the bug lives, and the test runner replaces their judgement of whether it is fixed. But no stage re-supplies the security review, because no stage has an oracle for it, and the 45% figure is what that missing oracle costs. Human-in-the-loop review (OpenHands, Devin) remains the default safeguard, but it reintroduces exactly the reading that vibe coding was defined by abandoning. The honest statement of the situation is that vibe coding, in the strict sense, is not yet a supported mode of operation; it is a mode of operation in which one has decided not to look.

6.5 Implications for Software Engineering Practice

These trends shift the developer's role rather than remove it. As agents absorb localisation, editing, and test-driven repair, the human contribution moves upstream (writing precise intent and acceptance criteria) and downstream (reviewing, verifying, and accepting patches). Two consequences follow from the evidence above. First, *verification becomes the bottleneck*: with a known security flaw in 45% of generation tasks [34] and benchmark scores partly reflecting reward hacking [8], organisations cannot treat a green test run as acceptance, and must invest in stronger oracles (security scanning, regression suites, and human review) as first-class pipeline stages. Second, *execution infrastructure becomes central*: sandboxes, reproducible environments, and CI integration are what make execution-grounded selection and refinement possible in the first place. The economic frame is shifting too, from "can an agent resolve this issue?" to "at what cost, and with what assurance?"

7 Conclusion

This report surveyed autonomous AI agents for code generation through a single lens, the five-stage vibe coding pipeline (intent, planning, generation, execution, refinement), and synthesised three active research fronts that instantiate its stages. From end-

to-end systems (§3) we found a convergent architecture: an observe–think–act loop over a purpose-built agent–computer interface, with repository-scale context managed by retrieval, AST search, or compression. Agentless shows that a fixed localise–repair–validate pipeline can rival interactive agents at lower cost, so autonomy is a design choice, not a prerequisite. From planning and search (§4) we found that what planning contributes is not deliberation but *grounding*: localisation hypotheses and candidate search anchored in program structure, which is why the least deliberative system remains competitive. From execution feedback (§5) we found that running the code, not generating it, supplies most of the reliability, and that knowing *when to stop* is as important as knowing how to fix.

The pipeline framing makes the field's central coupling visible: planning and refinement, usually studied in isolation, together form a feedback control layer that keeps generation tethered to external evidence. The framing also sharpens the claim to something falsifiable. That control layer exists only where an external oracle can contradict the model; internal self-critique, once budget-normalised, is an open loop wearing a control loop's clothes (§5.3). Read this way, the pipeline's two structural weaknesses are the two stages with no oracle behind them: intent capture, which has none at all (§4.1), and validation, whose oracle is weak enough to be gamed (§6.2). The same coupling exposes the limits surveyed in §6.2–§6.4: a saturating and contamination-prone evaluation landscape, a persistent context bottleneck, escalating cost, and a trust gap in which 45% of generation tasks yield code with a known security flaw [34]. The question at the frontier has changed accordingly. It is no longer "can an agent resolve real issues?" but "at what cost, and how much can the result be trusted?", and contamination-resistant benchmarks such as SWE-bench Pro [9], efficiency-aware scoring [15], and reward-hacking audits [8] are the instruments the field built to answer it.

The promising directions follow from that, and each amounts to supplying an oracle where one is missing. On evaluation, the community needs harder, multilingual, contamination-resistant benchmarks reported with cost and difficulty stratification, and benchmarks that score the pipeline's intermediate stages rather than only its final patch. On capability, long-horizon multi-file reasoning appears to be a training problem rather than a retrieval one (§6.4), and active context management the binding constraint on reachability. On specification, an agent that detects its own underspecified intent and asks, rather than guessing, would close the one stage that no system currently closes. On deployment, security-aware refinement objectives and well-designed human-in-the-loop control are needed before autonomous code generation can be trusted in production. Vibe coding has clearly crossed from autocomplete to autonomous issue resolution. Whether it crosses from impressive to dependable will be decided by whether the pipeline learns to say no to itself as reliably as a reader once did.

References

[1] AgentMarketCap. SWE-bench pro vs verified: The gap that rewrote 2026 agent procurement. https://agentmarketcap.ai/blog/2026/04/16/swe-bench-pro-vs-verified-25-point-gap-procurement-frame 2026. Paired Verified/Pro leaderboard data, compiled April 2026. Accessed: 2026-06-30.

[2] AgentMarketCap. SWE-bench verified: How AI coding agents went from 1.96% to 80.9% in 30 months. https://agentmarketcap.ai/blog/2026/04/09/swe-bench-verified-progress-timeline-2023-2026, 2026. Accessed: 2026-06-30.

[3] Answer.AI. Thoughts on a month with Devin. https://www.answer.ai/posts/2025-01-08-devin.html, 2025. Accessed: 2026-07-09.

[4] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.

[5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

[6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024.

[7] Cognition AI. Introducing devin, the first AI software engineer. https://cognition.ai/blog/introducing-devin, 2024. Accessed: 2026-07-09.

[8] Cursor. Reward hacking is swamping model intelligence gains. https://cursor.com/blog/reward-hacking-coding-benchmarks, 2025. Accessed: 2026-06-30.

[9] Xiang Deng et al. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[10] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. A survey on code generation with LLM-based agents. *arXiv preprint arXiv:2508.00083*, 2025.

[11] Epoch AI. What skills does SWE-bench verified evaluate? https://epoch.ai/publications/what-skills-does-swe-bench-verified-evaluate, 2025. Accessed: 2025-06-01.

[12] others. Context as a tool: Context management for long-horizon SWE-agents. *arXiv preprint arXiv:2512.22087*, 2025.

[13] others. Is vibe coding safe? benchmarking vulnerability of agent-generated code in real-world tasks. *arXiv preprint arXiv:2512.03262*, 2025.

[14] others. More with less: An empirical study of turn-control strategies for efficient coding agents. *arXiv preprint arXiv:2510.16786*, 2025.

[15] others. SWE-effi: Re-evaluating software AI agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*, 2025.

[16] Jatin Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html, 2025. Blog post, accessed: 2025-06-01.

[17] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024.

[18] Shuyang Jiang, Yuhao Wang, and Yu Wang. SelfEvolve: A code evolution framework via large language models. *arXiv preprint arXiv:2306.02907*, 2023.

[19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

[20] Andrej Karpathy. There's a new kind of coding i call "vibe coding". Post on X, 2 February 2025. https://x.com/karpathy/status/1886192184808149383, 2025. Accessed: 2026-07-09.

[21] Matous Kozak, Roshanak Zilouchian Moghaddam, and Kalpathy Sivaraman. When developer aid becomes security debt: A systematic analysis of insecure behaviors in LLM coding agents. In *Workshop on Security of Agentic AI (SEA), NeurIPS*, 2025.

[22] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[23] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[24] Nam Le Hai Nam et al. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[25] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *The Twelfth International Conference on Learning Representations*, 2024.

[26] OpenAI. Introducing SWE-bench verified. https://openai.com/index/introducing-swe-bench-verified/, 2024. Accessed: 2025-06-01.

[27] OpenAI. Why we no longer evaluate on SWE-bench verified. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/, 2026. Accessed: 2026-06-30.

[28] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. https://www.openhands.dev/blog/

sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model, 2025. Accessed: 2026-05-28.

[29] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-gym. *arXiv preprint arXiv:2412.21139*, 2024.

[30] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[31] SWE-bench Team. SWE-bench multilingual. http://www.swebench.com/multilingual.html, 2025. 300 tasks, 9 languages, 42 repositories. Accessed: 2026-06-30.

[32] SWE-bench Team. SWE-bench leaderboards. https://www.swebench.com. Underlying evaluation artefacts at https://github.com/SWE-bench/experiments, 2026. Accessed: 2026-07-09.

[33] various. Dissecting the SWE-bench leaderboards: Profiling submitters and architectures of LLM- and agent-based repair systems. https://arxiv.org/abs/2506.17208, 2025. arXiv:2506.17208.

[34] Veracode. 2025 GenAI code security report. https://www.veracode.com/blog/genai-code-security-report/, 2025. 45% of generation tasks introduce a known security flaw. Accessed: 2026-06-30.

[35] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.

[36] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024. ICLR 2025.

[37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[38] Yanlin Wang et al. Agents in software engineering: Survey, landscape, and vision. *arXiv preprint arXiv:2409.09030*, 2024.

[39] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[40] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

[41] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[42] Hao Xu et al. Aligning LLMs for FL-free program repair. *arXiv preprint arXiv:2404.08877*, 2024.

[43] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[44] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023.

[46] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.